

Trabalho Prático 2: Rede Social

Victor Romano Arcuri – 2025051241

Departamento de Ciência da Computação – Universidade Federal de
Minas Gerais (UFMG)
Belo Horizonte – MG – Brazil

victorromanoarcuri@ufmg.br

1. Introdução

O trabalho prático apresenta o cenário da ampliação da rede social acadêmica da plataforma ConectaDCC. Para tanto, faz-se necessário a criação de estruturas de Grafo que comportem Usuários e seus Temas de interesse, com enfoque nas relações Usuário-Usuário e Usuário-Tema. O objetivo central do trabalho consiste em comparar as duas abordagens principais para armazenamento de dados em Grafos: o armazenamento de uma Matriz de Adjacência e o armazenamento de Listas de Adjacência. Ademais, também é proposto, de maneira optativa, uma terceira abordagem de armazenamento, utilizando Listas de Arestas. Juntamente ao armazenamento das relações, diversas operações sobre os Grafos foram solicitadas, como inserção de nós, remoção de arestas, busca de vizinhos de um nó, e outras, permitindo operações básicas de redes sociais, como seguir usuários, deixar de seguir, verificar temas de interesse de usuários, etc. Por fim, uma análise experimental permite comparar os desempenhos de cada abordagem sobre as diferentes operações, em casos de teste específicos, como será detalhado adiante.

2. Método

2.1 Ambiente de Implementação e Execução

O código-fonte desenvolvido e os subsequentes testes executados na análise experimental foram realizados em um desktop com as seguintes especificações relevantes:

- ❖ **Memória:** 1 pente de 16 GB de memória RAM DDR4
- ❖ **Processador:** Intel® Core™ i5-10400F com 6 núcleos e 12 threads
- ❖ **Sistema Operacional:** Arch Linux x86_64
- ❖ **Linguagem:** C++ 11
- ❖ **Compilador:** G++ - GNU Compiler Collection

2.2 Arquitetura Principal

O projeto é estruturado de maneira que a implementação e cabeçalho de tipos abstratos de dados, TADs, são divididos em arquivos separados, localizados nos diretórios `include/` e `src/`, respectivamente. O arquivo com o método central `main()`, nomeado `main.cpp`, encontra-se em `src/`, sem cabeçalho correspondente, e é responsável por gerir

o fluxo de input de usuário, instanciar classes importadas e realizar as chamadas para os métodos necessários para gerir a lógica central do programa.

2.3 Tipos Abstratos de Dados

2.3.1 Lista

O TAD **Lista** é um invólucro das funcionalidades padrões de vetor, permitindo que um número ilimitado de elementos do mesmo tipo seja inserido, ao alocar dinamicamente tamanho conforme atinge sua capacidade máxima. Seus métodos permitem operações básicas de inserção, remoção, busca, etc.

2.3.2 Usuario e Tema

Os TADs **Usuario** e **Tema** representam as entidades básicas da rede social. O primeiro armazena id, nome e idade do usuário; o segundo armazena nome e tipo de um tema de interesse. Ambos expõem apenas métodos getters e setters de implementação direta.

2.3.3 MatrizAdjacencia

O TAD **MatrizAdjacencia** representa a estrutura principal de armazenamento de dados para Grafos na abordagem de Matriz de Adjacência, representada pela letra M. Esse TAD armazena a variável *dimensoes* do tipo `int`, que representa o número de dimensões dessa Matriz de Adjacência (por padrão, duas), e uma variável do tipo `Lista<Lista>` chamada *dados*, representando uma matriz 2x2 em que cada posição `[i][j]` equivale a 1 se existe uma aresta de *i* para *j*, e 0 caso contrário. Seus métodos principais são:

- ❖ **aumentar_dimensao()**: Expande simetricamente a grade da matriz adicionando uma nova linha e coluna preenchidas com zeros para representar as potenciais conexões de um novo nó.
- ❖ **alterar_valor()**: Grava a existência ou remoção de uma aresta inserindo o status correspondente (0 ou 1) no cruzamento exato das coordenadas especificadas.
- ❖ **obter_valor()**: Retorna o status (0 ou 1) de uma conexão, lendo a célula `[id1][id2]` referenciada.

2.3.4 ListaAdjacencia

O TAD **ListaAdjacencia** representa a estrutura principal de armazenamento de dados para Grafos na abordagem de Listas de Adjacência, representada pela letra L. Depende do struct **No**, o qual representa um vértice do grafo, armazenando seu tipo (**USUARIO** ou **TEMA**), seu identificador dentro do grafo e o identificador referente ao sistema. Armazena duas variáveis principais: um `int` *tamanho*, que representa o número de vértices do Grafo, e `Lista<Lista<No >>` *nos*, que representa uma lista em que cada índice se refere a um vértice, e possui uma lista de **No** referente a quem esse nó se liga. Seus métodos principais são:

- ❖ **aumentar_tamanho()**: Cria e anexa uma nova lista sequencial vazia, provisionando espaço para comportar os vizinhos de um vértice recém-adicionado.

- ❖ `adicionar_aresta()`: Insere o ponteiro do nó de destino na sublista de vizinhança pertencente ao nó de origem, criando uma conexão entre eles.
- ❖ `remover_aresta()`: Vasculha e varre as referências, desvinculando um nó vizinho específico da lista de conexões de um determinado vértice.

2.3.5 ListaArestas

O TAD `ListaArestas` atua como um registro global centralizado, armazenando estritamente as relações ativas e descartando a reserva de espaço vazio, sendo a opção voltada para redes muito esparsas. Seus métodos principais são:

- ❖ `adicionar_aresta()`: Define um novo par de IDs de origem e destino, catalogando a direção na estrutura relacional principal.
- ❖ `remover_aresta()`: Procura e desativa a correlação registrada atrelada ao par especificado.
- ❖ `obter_destinos_de()` e `obter_origens_de()`: Entregam um conjunto listado contendo de onde vieram ou para onde vão as setas de um nó específico.

2.3.6 Grafo

O TAD `Grafo` representa a estrutura de dados Grafo, armazenando vértices e arestas para relacionar dados, os quais podem ser armazenados em Listas de Adjacência, Matriz de Adjacência ou Lista de Arestas. O TAD possui uma variável para armazenar seus vértices, `Lista<No*>` nos, uma para armazenar o seu tipo (`GRAFO_MATRIZ`, `GRAFO_LISTA`, `GRAFO_ARESTA`), `TipoGrafo tipo`, e uma variável para armazenar os dados de cada tipo: `MatrizAdjacencia *matriz; ListaAdjacencia *lista; ListaArestas *lista_arestas`. Seus métodos principais são:

- ❖ `Grafo()` e `~Grafo()`: Rotinas de inicialização e limpeza da estrutura base e alocação dinâmica dos TADs correspondentes ao tipo de armazenamento escolhido.
- ❖ `retornar_no_por_id_interno()`: Acessa a lista principal de vértices e resgata o ponteiro exato do nó correspondente ao índice do grafo submetido.
- ❖ `criar_no()`: Instancia um novo vértice e o acopla à topologia local, atualizando as dimensões da base de armazenamento selecionada.
- ❖ `criar_aresta()`: Cria a conexão entre dois nós, de maneira unidirecional ou bidirecional, chamando os métodos respectivos ao tipo de método de armazenamento utilizado.
- ❖ `trocar_tipo()`: Modifica o tipo de armazenamento utilizado, transcrevendo os nós e arestas de um tipo para outro, como por exemplo, partindo do uso de uma Matriz de Adjacência, para o uso de Listas de Adjacência, removendo a implementação anterior de modo livre de vazamento de memória.
- ❖ `obter_vizinhos_apontados_por()`: Filtra a estrutura de dados para devolver todos os nós apontados por um vértice de origem.
- ❖ `obter_vizinhos_que_apontam_para()`: Varre a topologia inversamente para relatar quais nós têm conexão direcionada de entrada para o vértice alvo.

- ❖ `obter_vizinhos_bidirecional()`: Cruza conexões de saída com as de entrada para reportar apenas nós com setas em ambas as direções simultaneamente.
- ❖ `checar_aresta()`: Verifica de maneira pontual se existe uma conexão explícita do nó de origem para o nó de destino.
- ❖ `remover_aresta()`: Procura e desfaz a ligação existente entre os pares informados, espelhando a exclusão caso sinalizado como bidirecional.

2.3.7 Dicionario

O TAD **Dicionario** atua como camada de tradução entre os identificadores externos do sistema, fornecidos sequencialmente pela entrada, como 0, 1, 2..., e os índices internos (`id_grafo`) alocados independentemente por cada estrutura de grafo, evitando buscas exaustivas.

Para o **grafo social**, o **Dicionario** utiliza a lista `map_usuario_social`, onde o índice corresponde ao ID externo do usuário e o valor armazenado é o seu ID interno nesse grafo. Os métodos `registrar_usuario_social()` e `get_no_usuario_grafo_social()` inserem e consultam esse mapa em $O(1)$ por indexação direta.

Para o **grafo de temas**, a numeração interna mistura usuários e temas em uma mesma contagem, exigindo dois mapas separados: `map_usuario_temas` e `map_tema_temas`. Cada um associa o ID externo da entidade ao seu ID interno nesse grafo, e os métodos correspondentes (`registrar_usuario_temas()`, `registrar_tema_temas()`, `get_no_usuario_grafo_temas()`, `get_no_tema_grafo_temas()`) operam pelo mesmo princípio de acesso em $O(1)$.

Por fim, o **Dicionario** armazena referências às listas brutas de cadastro gerenciadas pelo **Sistema**, permitindo que `get_usuario()` e `get_tema()` retornem os atributos brutos de uma entidade diretamente pelo ID externo, sem interagir com a lógica dos grafos.

2.3.8 Sistema

Atua como orquestrador global, mantendo o **Dicionario** e os grafos (social e temático), além de processar o protocolo de entrada e saída. Suas responsabilidades dividem-se em:

- ❖ **Gerenciamento e Cadastro:** Métodos como `adicionar_usuario`, `adicionar_tema`, `seguir` e `remover_seguimento` (comandos U, T, S, R) atualizam a base de dados, os vértices e as relações nos grafos;
- ❖ **Consultas e Buscas:** Agrupa comandos de vizinhança (LT, LC, LS, LA) e status (Q, G, F), utilizando o algoritmo *QuickSort* adaptado internamente para garantir as saídas ordenadas exigidas no protocolo;
- ❖ **Recomendação Avançada:** Implementa a operação bônus (P), combinando uma travessia em largura (BFS) para cálculo de distâncias com a métrica de afinidade temática de Jaccard, finalizando com um *Insertion Sort* seletivo e tolerante a falhas de precisão flutuante para elencar os *top-K* candidatos.

3. Análise de Complexidade

Nesta seção, avalia-se o custo assintótico de tempo e espaço computacional das estruturas de dados e dos principais algoritmos desenvolvidos. Para fins de notação, as seguintes variáveis serão utilizadas: V como o número de vértices (nós correspondentes a Usuários ou Temas), E como o número de arestas (relações sociais ou interesses temáticos) e d como o grau médio de conexões de um nó.

3.1. Estruturas Base: Lista e Dicionário

A estrutura Lista é a fundação para a dinamicidade de armazenamento do sistema. Como ela aloca memória de forma contígua e a duplica (`redimensionar()`) apenas quando a capacidade máxima é atingida, a inserção no final (`inserir()`) e o acesso direto via índice (`obter()`) possuem complexidade de tempo de $O(1)$ amortizado e $O(1)$ estrito, respectivamente. Por outro lado, as operações `encontrar()` (busca linear) e `remove()` (que exige o deslocamento dos elementos subsequentes) operam em tempo $O(N)$, onde N é o número de elementos na lista. O espaço em memória ocupado é $O(N)$.

O Dicionário usufrui da natureza amortizada da Lista para manter mapas de acesso em $O(1)$.

- ❖ **Tempo:** Os métodos de cadastro (`registrar_usuario_social`, etc.) e de tradução (`get_no_usuario_grafo_social`, etc.) utilizam a indexação direta do vetor. Logo, a tradução do identificador externo para o interno ocorre em tempo constante de $O(1)$.
- ❖ **Espaço:** Armazena um inteiro para cada usuário e tema de domínio, resultando em uma complexidade espacial de $O(V)$.

3.2. Abordagens de Armazenamento do Grafo

A diferença de desempenho na gestão de redes sociais depende estritamente da estrutura de armazenamento selecionada pelo Sistema.

- ❖ **Matriz de Adjacência (MatrizAdjacencia):**
 - **Espaço:** A matriz aloca uma grade simétrica para relacionar todos os nós, possuindo complexidade espacial quadrática de $O(V^2)$, o que a torna custosa para redes muito esparsas.
 - **Adicionar Nó (criar_no):** Ao inserir um vértice, a dimensão da matriz precisa crescer. Anexar uma célula nula a cada linha existente e criar uma nova linha inteira exige um esforço computacional de $O(V)$.
 - **Arestas (Criar, Checar, Remover):** O acesso cruzado `[id1][id2]` garante tempo constante de $O(1)$.
 - **Recuperação de Vizinhos:** Obter quem aponta para um nó ou para quem ele aponta (`obter_vizinhos`) exige varrer uma linha ou coluna inteira da matriz de dimensões V . A complexidade é sempre $O(V)$, não importando se o usuário segue 1 ou 1000 pessoas.
- ❖ **Listas de Adjacência (ListaAdjacencia):**
 - **Espaço:** Armazena apenas os nós e as relações efetivamente existentes. A complexidade de espaço é proporcional à densidade da rede, dada por $O(V + E)$.

- **Adicionar Nó (criar_no):** Instanciar uma lista de vizinhos vazia ao final do array base é $O(1)$ amortizado.
- **Arestas (Criar, Checar, Remover):** Inserir é $O(1)$. Contudo, verificar a existência de uma relação ou removê-la exige percorrer a sublista de vizinhos do nó de origem, operando em tempo de $O(d)$, onde d é o grau de saída do nó.
- **Recuperação de Vizinhos:** Obter as relações de saída (obter_vizinhos_apontados_por) exige apenas varrer a sublista local, operando eficientemente em $O(d)$. Entretanto, para achar os seguidores (obter_vizinhos_que_apontam_para), o algoritmo é obrigado a varrer todas as listas de todos os vértices buscando o ID alvo, resultando em um custo de $O(V + E)$.
- ❖ **Lista de Arestas (ListaArestas):**
 - **Espaço:** Focada no minimalismo extremo, armazena apenas as ligações. Ocupa espaço de $O(E)$.
 - **Adicionar Nó (criar_no):** Custo interno irrelevante, operando em $O(1)$.
 - **Arestas (Criar, Checar, Remover) e Recuperação:** Qualquer busca (para saber quem segue, quem é seguido, ou para desfazer conexões) demanda varrer o vetor principal de forma exaustiva. Logo, a ampla maioria das consultas nessa estrutura é fortemente penalizada, possuindo tempo atrelado à quantidade total de arestas da rede em $O(E)$.
- ❖ **Migração de Estruturas (trocar_tipo):**
 - A funcionalidade que converte o grafo sem perda de dados depende do estado de origem e de destino. Migrar para uma Matriz custa invariavelmente $O(V^2)$, pois requer alocar e inicializar toda a grade. Migrar de uma Matriz para Listas custa os mesmos $O(V^2)$, visto que é preciso ler cada quadrante para detectar relações. Alterações entre Lista de Adjacência e Lista de Arestas são mais baratas e escalam em função dos elementos reais, operando em $O(V + E)$.

3.3. Algoritmos Orquestradores do Sistema

O Sistema coordena as respostas baseando-se no processamento entregue pelo Grafo.

- ❖ **Consultas de Listagem (Temas, Seguidores, Seguidos e Amigos):**

O custo total destas requisições corresponde à soma da busca de vizinhos do grafo atual com a ordenação lexicográfica de exibição. O Sistema utiliza uma variação interna do algoritmo *QuickSort* (quickSort_internos), que atua sobre o subconjunto recuperado. Sendo K o número de nós encontrados na consulta (o tamanho real da vizinhança devolvida), o custo da ordenação no caso médio é de $O(K \log K)$.
- ❖ **Operação de Recomendação (Operação bônus P):**

Este é o fluxo algorítmico mais complexo da aplicação.

 - A função consultar_recomendacao inicializa calculando a menor distância do alvo a todos os demais usuários no grafo social executando uma Busca em Largura (BFS). O custo da BFS acompanha a forma de acesso à

vizinhança: se em Matriz custa $O(V^2)$, em Lista de Adjacência opera em $O(V + E)$.

- Em seguida, interage sobre todos os outros vértices, computando a similaridade de interesses (`calcular_jaccard`). O cálculo de intersecção e união no Jaccard possui custo de $O(T_u * T_v)$, onde T é a quantidade de temas das duas partes.
- Para finalizar, a inserção e ordenação no array restrito de candidatos para os "top-K" é sustentada por uma ordenação simples estável (*Insertion Sort*), que emula pontuações fracionárias sem discrepâncias devido ao limitador de precisão flutuante. O pior caso da ordenação de todos os V elementos encontrados é $O(V^2)$. Portanto, a rotina bônus é naturalmente pesada e apresenta complexidade assintótica dominante em função do seu algoritmo de ordenação de candidatos e travessia da rede.

4. Estratégias de Robustez

Foram aplicadas técnicas de programação defensiva em diferentes camadas da arquitetura. `MatrizAdjacencia` e `Lista` validam limites antes de qualquer acesso, lançando exceções (`std::out_of_range`, `std::invalid_argument`) para abortar operações inválidas antes que ocorra corrupção de memória. O `Grafo` protege acessos indevidos retornando `nullptr` em falhas internas; o `Dicionario`, ao receber esse ponteiro nulo, lança exceção antes que o `Sistema` tente desreferenciá-lo. Em `ListaAdjacencia`, `remover_aresta()` verifica previamente a existência do nó destino e retorna silenciosamente caso não o encontre, blindando o fluxo contra comandos redundantes. Por fim, métodos que envolvem divisão, como `calcular_jaccard` e o cálculo de proximidade da recomendação, possuem validação explícita para evitar divisão por zero.

5. Análise Experimental

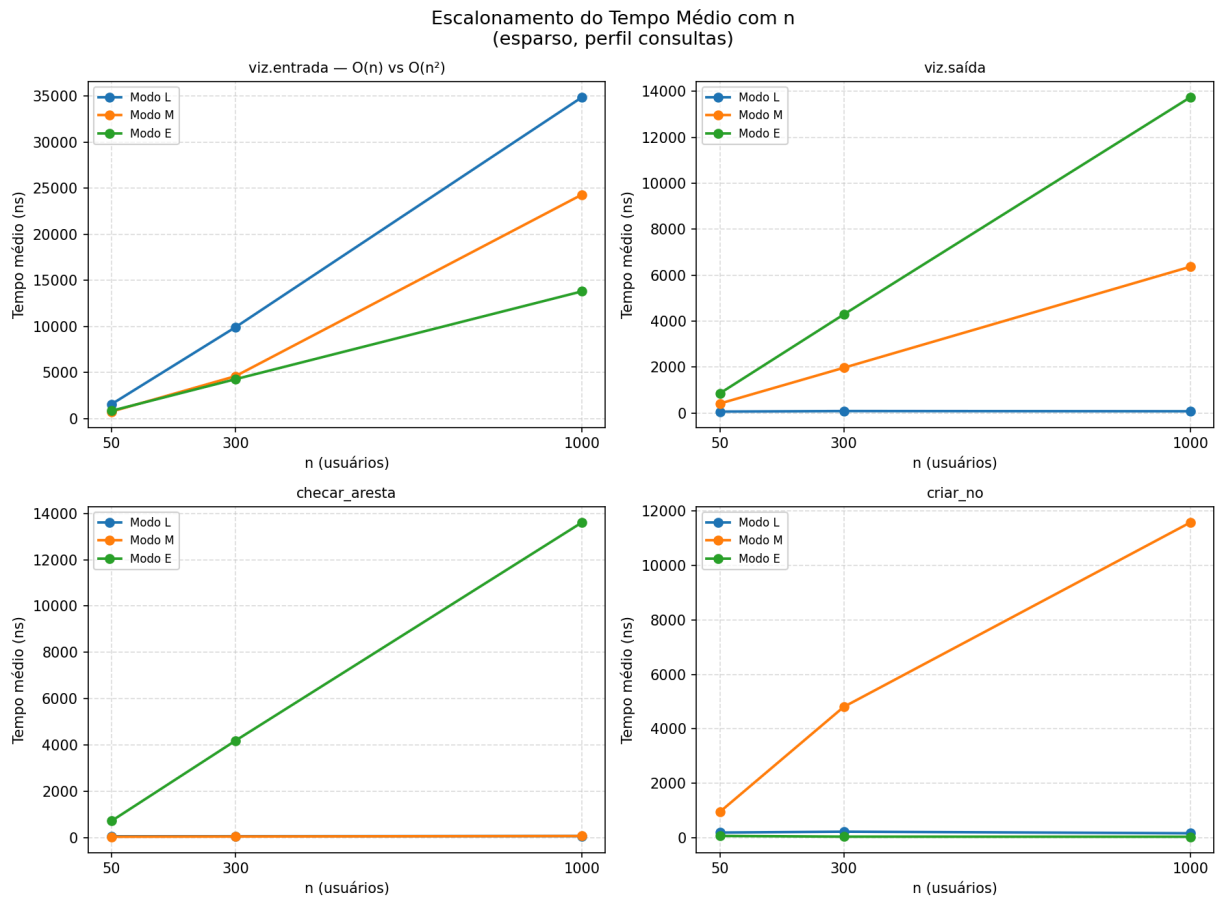
Para avaliar o desempenho prático das diferentes estruturas de armazenamento implementadas (Lista de Adjacência - **L**, Matriz de Adjacência - **M** e Lista de Arestas - **E**), foi desenvolvida um conjunto de casos de testes que simulam o uso real da plataforma ConectaDCC. Os experimentos variaram três dimensões principais: o **tamanho da rede** (n variando de 50 a 1000 usuários), a **densidade de conexões** (redes esparsas e as densas) e o **perfil de carga** (lotes de entrada focados em atualizações e os focados em consultas).

Os tempos de execução foram isolados internamente utilizando a biblioteca `<chrono>`, garantindo que o gargalo de leitura e escrita do terminal não interferisse nas medições.

5.1. Escalonamento e Comportamento Assintótico

Para validar a complexidade teórica das estruturas, mediu-se o tempo de execução de operações individuais à medida que a rede crescia.

O **Grafo 01** ilustra o escalonamento do tempo médio das operações do Grafo Social em função do número de usuários na rede. A partir de sua análise, confirmam-se os seguintes comportamentos práticos:



Grafo 01 - Escalonamento do Tempo Médio com n.

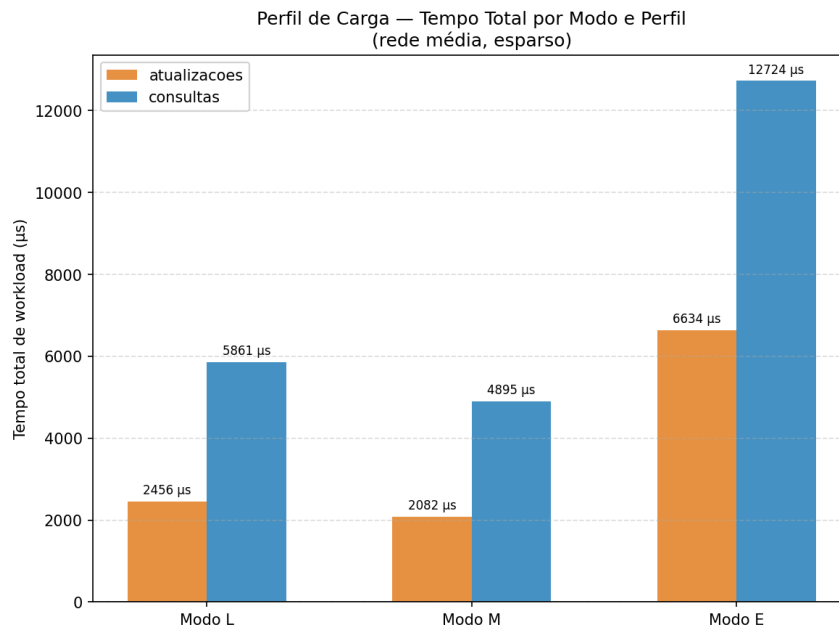
- ❖ **Consulta de Vizinhos de Saída (viz.saída):** A Lista de Adjacência (Modo L) manteve um tempo de resposta praticamente constante e inferior a todos os outros métodos, refletindo sua eficiência em buscar diretamente a sublista do usuário. Em contrapartida, a Matriz (Modo M) apresentou crescimento estritamente linear, punida pela necessidade de varrer toda uma linha de tamanho V mesmo quando o usuário segue poucas pessoas.
- ❖ **Criação de Nós (criar_no):** Fica evidente o alto custo de manutenção da Matriz de Adjacência. Enquanto os modos L e E mantêm tempos virtualmente colados a zero (tempo constante), o modo M exhibe uma reta ascendente acentuada, causada pelo custo de realocar espaço e adicionar novas dimensões preenchidas com zeros à grade matricial bidimensional.
- ❖ **O Gargalo da Lista de Arestas:** Em todas as operações de busca representadas no Grafo 01 (como `checar_aresta` e `viz.entrada`), a Lista de Arestas (Modo E) apresentou o pior desempenho escalável, com uma inclinação de degradação

agressiva. A necessidade de varrer o registro global de ponta a ponta inviabiliza essa estrutura para operações de leitura.

5.2. Impacto do Perfil de Carga e Uso Prático

Para entender como as estruturas se comportam sob estresse no sistema orquestrador, avaliou-se o tempo total de processamento de lotes de comandos.

O Grafo 02 compara o desempenho total do sistema sob dois perfis de carga em uma rede média esparsa.



Grafo 02 - Tempo Total por Modo e Perfil.

No perfil de **consultas** (80% recomendações e listagens), a Matriz de Adjacência obteve o menor tempo total (4.895 µs contra 5.861 µs da Lista), beneficiada pelo acesso $O(1)$ em `checar_aresta`, operação central de algoritmos como *BFS* e interseção de amigos mútuos. No perfil de **atualizações** (75% seguir/deixar de seguir), os modos M e L foram competitivos (2.082 µs vs 2.456 µs), indicando que a Matriz é vantajosa sempre que o consumo de memória não for impeditivo. O Modo E apresentou os piores tempos nos dois perfis (12.724 µs e 6.634 µs), reforçando sua limitação a redes estáticas com restrições extremas de memória.

5.3. Efeito da Densidade Topológica

A densidade social da rede apresentou impactos assimétricos dependendo da estrutura subjacente. Ao observar as métricas de tempo de `vizinhos_apontam_para` e `vizinhos_apontados_por`, nota-se que o tempo de execução da Lista de Adjacência dobra ou triplica ao transitar de uma rede "esparsa" para "densa", visto que a quantidade de ponteiros armazenados nas sublistas aumenta significativamente. A Matriz de Adjacência manteve-se completamente imune e estável perante o aumento de densidade,

pois seu algoritmo independe do preenchimento da grade (ela sempre inspeciona V células, independentemente de guardarem 0 ou 1).

6. Conclusões

Este trabalho prático abordou a modelagem e a implementação do núcleo de uma rede social acadêmica por meio do desenvolvimento de Tipos Abstratos de Dados baseados em Grafos, suportando múltiplas representações de memória de forma dinâmica. A partir da construção do sistema e da condução da análise experimental, foi possível compreender na prática os compromissos entre complexidade de tempo e de espaço inerentes a cada abordagem de armazenamento.

Os resultados evidenciaram que a eficiência de uma estrutura não é absoluta, mas sim fortemente dependente do domínio da aplicação. Constatou-se que a Lista de Adjacência é altamente performática para varreduras de vizinhança direta, sendo a escolha ideal para redes esparsas e com alto volume de novos cadastros, embora sofra degradação de desempenho conforme a densidade topológica aumenta. Em contrapartida, a Matriz de Adjacência, apesar do custo de espaço quadrático e da penalidade estrutural ao instanciar novos nós, demonstrou superioridade notável em cenários de estresse dominados por consultas complexas — como algoritmos de recomendação —, mantendo sua velocidade de acesso constante e inabalável perante o adensamento da rede. Por fim, a Lista de Arestas provou-se impraticável para redes de alta dinamicidade, possuindo utilidade restrita a ambientes com severa limitação de memória.

Em suma, o desenvolvimento deste projeto consolidou o aprendizado prático sobre gestão de memória e encapsulamento em C++, além de demonstrar a extrema importância de se realizar uma análise de complexidade prévia e medições instrumentadas para guiar a escolha da estrutura de dados mais adequada ao perfil de carga de um sistema no mundo real.

6. Bibliografia

Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2012). *Algoritmos: Teoria e Prática* (3ª ed.). Elsevier.

Ziviani, N. (2006). *Projeto de Algoritmos com implementações em Java e C++*. Thomson Learning.